

Aspect-Based Sentiment Analysis (ABSA) – Project Report

Vladalin Bosii and Aleksiya Solovyova, class 302

1. Introduction: Problem description and project goals

Aspect-Based Sentiment Analysis (ABSA) aims to identify specific aspects of an entity mentioned in text (e.g., food, service, ambiance in a restaurant review) and determine the sentiment expressed toward each aspect individually. Unlike document-level or sentence-level sentiment, ABSA produces fine-grained outputs such as (aspect, sentiment, confidence), optionally with text spans.

Goal of this project: Implement and compare multiple ABSA approaches with a shared interface so that they can be evaluated on a common dataset and used interchangeably in downstream tasks.

Key outputs per input text:

- aspect: A token or phrase representing the discussed facet (e.g., “service”).
- sentiment: A categorical label (typically positive, negative, neutral).
- confidence: A floating-point estimate of certainty for the prediction.

2. Methodology: Detailed explanation of each implementation

The project implements three approaches (see `src/`):

2.1. LexiconABSA (`src/lexicon_absa.py`)

- Core idea: Use rule-based NLP with spaCy for linguistic processing and NLTK VADER for sentiment polarity.
- Processing steps:
 - Tokenization, POS tagging, and dependency parsing to discover candidate aspect terms (nouns and noun chunks).
 - Use VADER sentiment scores around aspect mentions to infer polarity.
 - Heuristics for negation handling and noise filtering (e.g., ignore stopwords, short tokens).

2.2. TransformerABSA (`src/transformer_absa.py`)

- Core idea: Fine-tune or use a pretrained transformer model (Hugging Face) to classify aspect-level sentiment.
- Processing steps:
 - Use a transformer encoder to embed text; optionally detect aspects via noun chunks or predefined schema.
 - Classify sentiment for each aspect span; compute softmax probabilities as confidence.

2.3. LLMABSA (`src/llm_absa.py`)

- Core idea: Prompt a local Large Language Model via Ollama to extract structured JSON of aspects and sentiments.
- Processing steps:
 - Carefully crafted system/user prompts instruct the model to output JSON with fields: aspect, sentiment, confidence, optional span.
 - Parse and validate JSON with fallbacks and minimal post-processing.

Common interface

- Base class `ABSAAnalyzer` in `src/base.py` defines `analyze(text) -> List[AspectSentiment]`.
- `AspectSentiment` data class standardizes outputs across implementations.
- Utilities in `src/utils.py` load test samples and showcase the prototype for model evaluation.

3. Design Decisions: Why certain choices were made

3.1. Lexicon Implementation

- For extraction, implemented a union strategy to maximize recall while preventing duplicates
- For Opinion-Aspect Linking, I tried a 3-tier strategy, starting with dependency-based, proximity-based as a fallback and finally sentence-level.
- The trade-off is that it heavily struggles with implicit aspects and sarcasm.

3.2. Transformer Implementation

- Model Selection: Selected yangheng/deberta-v3-base-absa-v1.1 as it's already fine-tuned for ABSA tasks, eliminating the need to train a model from scratch on restaurant reviews.
- Aspect Extraction: Used spaCy to extract nouns (NOUN and PROPN tags) as aspect candidates. This two-stage approach separates finding aspects from analyzing sentiments. Limited to 5 aspects per review to keep processing manageable.
- Input Format: Formatted inputs as "text [SEP] aspect" which is what the model expects, it passes the full review text along with each specific aspect to get targeted sentiment predictions.
- Device Handling: Added GPU support with automatic fallback to CPU. The model automatically moves to GPU when available.
- Memory Optimization: Used `torch.no_grad()` during prediction to prevent gradient calculation, which isn't needed and wastes memory.
- Filtering: Excluded generic words like 'thing', 'place', 'way', 'time' and anything under 3 characters to avoid extracting meaningless aspects.
- Fallback: Returns 'overall' as a default aspect when no nouns are found, ensuring the model always produces output rather than returning empty results.
- The trade-off is that while it achieves strong sentiment classification accuracy, the aspect extraction is simplistic and will miss implicit aspects, multi-word phrases (e.g., "customer service"), and aspects that aren't strictly nouns, making it less comprehensive than the LLM approach but faster and more consistent than the lexicon method.

3.3. LLM Implementation

- Model Selection: Selected Qwen 2.5 7B after testing multiple models, as it provided the optimal balance of accuracy and inference speed on limited hardware

- while handling JSON output formatting reliably.
- Prompt Engineering: Implemented chain-of-thought prompting with explicit instructions, 4 few-shot examples, and structured JSON schema to maximize consistency and reduce hallucinations.
- Configuration: Set temperature=0.0 for reproducible results, top_k=10 for focused sampling, and num_predict=150 to limit response length to ~5-8 manageable aspects.
- JSON Parsing Strategy: Built robust multi-level error recovery including regex repair for common formatting issues (missing commas, trailing commas, markdown fences).
- The trade-off is highest accuracy and flexibility at the cost of significant computational overhead and processing time.

4. Experimental Setup: Data, evaluation metrics, test scenarios

Data

- Sample data lives in data/test_samples.json. It contains 100 sample reviews, inspired by the Yelp dataset.

Evaluation metrics

- Per-model classification quality: accuracy, precision, recall, F1.
- Extraction behavior: average number of aspects per review; qualitative correctness of spans.
- Runtime: average processing time per review.

Test scenarios

- Use the notebook notebooks/comparison.ipynb to:
 - Run each analyzer on the same dataset.
 - Collect per-sample predictions and timings.
 - Compute summary metrics and generate a performance comparison bar chart.
- Basic unit tests exist under tests/test_absa.py to validate core functionality.

5. Results & Analysis

5.1. Qualitative comparison (examples)

- LexiconABSA: Often extracts clear noun aspects (e.g., “service”, “food”) but can miss implicit aspects or mishandle negations in complex sentences. It is abysmal at sarcasm.
- TransformerABSA: Better at context-dependent polarity; can identify sentiments even with subtle cues; aspect detection quality depends on preprocessing.
- LLMABSA: Flexible at finding aspects and paraphrased sentiments; may produce extra or inconsistent aspects unless prompts/validators are strict.

5.2. Quantitative comparison

SUMMARY METRICS							
Model	Accuracy	Precision	Recall	F1 Score	Avg Time (s)	Avg Aspects	Avg Confidence
Lexicon	0.420	0.443	0.409	0.386	0.007	1.6	0.32
Transformer	0.620	0.613	0.603	0.595	0.134	2.3	0.92
LLM	0.740	0.748	0.736	0.739	12.684	1.8	0.84

5.3. Performance metrics (speed, resource usage)

- LexiconABSA: Fastest, CPU-only; minimal memory footprint.
- TransformerABSA: Moderate-to-slow on CPU; faster with GPU; higher memory use due to model weights.
- LLMABSA: Latency depends on model size and hardware; JSON validation adds minor overhead; Takes just under 20 minutes to process 100 samples, but can be faster on a more powerful machine.

6. Discussion

6.1. Strengths and weaknesses of each approach

- LexiconABSA
 - Strengths: Simplicity, speed, no training, stable outputs.
 - Weaknesses: Limited domain transfer; brittle to sarcasm, implicit aspects, and complex syntax.
- TransformerABSA
 - Strengths: Strong contextual understanding, probabilistic outputs, better generalization.
 - Weaknesses: Heavier dependencies, requires more compute/resources.
- LLMABSA
 - Strengths: Very flexible extraction, easy to adapt via prompting, good zero-shot performance.
 - Weaknesses: Output determinism and validation challenges; can hallucinate aspects; latency.

6.2. Use case recommendations

- Real-time, resource-constrained or on-device scenarios: Prefer LexiconABSA.
- Batch processing with quality priority and available compute: Prefer TransformerABSA.
- Rapid prototyping or diverse/unseen domains without labels: Prefer LLMABSA with strict JSON schema and validation.

7. Conclusion: Key learnings and potential improvements

Key learnings

- ABSA benefits from combining reliable extraction logic with robust sentiment classifiers.
- Shared interfaces and evaluation harnesses make comparisons straightforward.
- LLMs are powerful zero-shot extractors but require careful output control.

Potential improvements

Lexicon-Based:

- Domain-specific lexicon expansion for restaurant terminology
- Advanced negation handling for complex sentence structures
- Aspect grouping/normalization (e.g., "waiter" → "service")
- Integration of context-aware polarity reversal

Transformer-Based:

- Better aspect extraction, using multi-word phrase detection, NER, and implicit aspect detection
- Two-stage architecture application
- Minor performance, confidence and context window optimizations
- Integration of context-aware polarity reversal

LLM-Based:

- Integration of larger models (13B/70B) for improved understanding
- Custom few-shot examples tailored to specific domains
- Chain-of-thought reasoning for aspect justification
- Hybrid approaches combining LLM extraction with faster classification

Cross-Implementation:

- Multi-language support for international reviews
- Real-time feedback integration for model improvement
- Confidence interval estimation for prediction uncertainty
- Custom sentiment categories beyond positive/negative/neutral